QuillAudits

AUDIT REPORT

March 2025

For

Table of Content

Executive Summary 04

Number of Security Issues per Severity 06

Summary of Issues 07

Checked Vulnerabilities 08

Techniques and Methods 10

Types of Severity 12

Types of Issues 13

Severity Matrix 14

Critical Severity Issues 15

1. manualBurn breaks rate and contract-wide token accounting if the owner address is excluded from rewards 15

Medium Severity Issues 17

2. Trapped ETH in contract due to swap–and–liquify price impact mismatch 17

Informational Issues 18

3. Inconsistent naming convention for private functions 18

4. Unused return values in addLiquidity 18

5. Dead code in _tokenTransfer 19

6. Zero slippage protection in swap and liquidity calls 19

7. Uses string revert reasons instead of custom errors 20

8. Redundant SafeMath usage on Solidity 0.8.20 20

9. Missing events for fee and state changes 20

Functional Tests 21

Automated Tests 21

Threat Model 22

Closing Summary & Disclaimer 29

Executive Summary

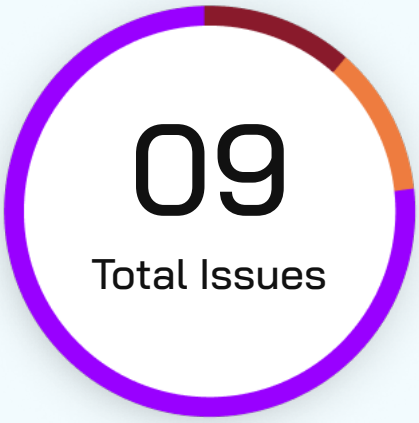
Project Name	GrowFitech
Protocol Type	Safemoon fork, burnable
Project URL	https://www.growfitech.com/
Overview	The LiquidityGeneratorToken is an advanced ERC20 token that combines reflection-based rewards, automated liquidity generation, and multi-tier fees to create a self-sustaining ecosystem. It implements a dual balance system (reflection and token balances) that automatically distributes transaction fees to all holders proportionally without requiring active participation, while simultaneously accumulating liquidity fees that trigger automated Uniswap V2 swaps and liquidity provision when thresholds are met. The contract enforces a 25% fee ceiling across three independent fee types (tax, liquidity, and charity), includes owner-controlled blacklist management to prevent problematic transfers, supports 72-hour cooldown manual token burns to reduce supply, and provides sophisticated exclusion mechanisms for both rewards and fees. Built with OpenZeppelin standards and integrated with Uniswap V2, the contract prioritizes security through reentrancy guards, supply overflow protection, and comprehensive event logging, while giving the owner significant control over fee structures, address exclusions, and token supply dynamics. The token's core strength lies in its ability to reward passive holders, maintain automatic liquidity, and support charitable contributions simultaneously, making it ideal for community-focused DeFi projects.
Audit Scope	The scope of this Audit was to analyze the GrowFitTech Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/ggauranshi-03/LiquidityGeneratorToken
Branch	main
Contracts in Scope	Token.sol
Commit Hash	c7dc2d
Language	Solidity

Blockchain	Polygon
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	4th February 2026 - 6th February 2026
Updated Code Received	28th February, 2026
Review 2	2nd March, 2026
Fixed In	https://github.com/ggauranshi-03/LiquidityGeneratorToken/blob/main/token_after_report.sol
Mainnet Address	https://polygonscan.com/address/0xBCCb07330C90E634373c8c26AbD81B18FB62543f#code

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	1 (11.1%)
High	0 (0.00%)
Medium	1 (11.1%)
Low	0 (0.00%)
Informational	7 (77.8%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	**4**
Partially Resolved	0	0	0	0	**1**
Resolved	**1**	0	**1**	0	**2**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	manualBurn breaks rate and contract-wide token accounting if the owner address is excluded from rewards	Critical	Resolved
2	Trapped ETH in contract due to swap–and–liquify price impact mismatch	Medium	Resolved
3	Inconsistent naming convention for private functions	Informational	Resolved
4	Unused return values in addLiquidity	Informational	Resolved
5	Dead code in _tokenTransfer	Informational	Acknowledged
6	Zero slippage protection in swap and liquidity calls	Informational	Partially Resolved
7	Uses string revert reasons instead of custom errors	Informational	Acknowledged
8	Redundant SafeMath usage on Solidity 0.8.20	Informational	Acknowledged
9	Missing events for fee and state changes	Informational	Acknowledged

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Critical Severity Issues

manualBurn breaks rate and contract-wide token accounting if the owner address is excluded from rewards

Resolved

Path

Token.sol

Function Name

`manualBurn()`

Description

If the owner is excluded (i.e. _isExcluded[owner()] == true), the function updates _rOwned[owner()] but not _tOwned[owner()]. This is problematic because _tOwned tracks the balance updates for excluded accounts, if this manual token burn is not reflected in _tOwned the reflection accounting invariant is irredeemably broken.

Impact

Incorrect balance calculations, unexpected rate skew and accounting corruption.

Likelihood

This scenario is guaranteed to happen as the owner's address is excluded from rewards as set in the constructor.

POC

If:

```
_tTotal = 1,000
_rTotal = 1,000,000 → rate = 1,000
```

Owner is excluded and has:

```
_tOwned[owner] = 100
_rOwned[owner] = 100,000 (consistent)
```

Manual burn of 50 tokens should make:

```
_tTotal = 950
_rTotal = 950,000
```

If excluded owner is handled correctly:

```
_tOwned[owner] = 50
_rOwned[owner] = 50,000
```

Rate still = 1,000

But current manualBurn only reduces _rOwned:

```
_tOwned[owner] stays 100
_rOwned[owner] becomes 50,000
_tTotal = 950, _rTotal = 950,000
```

Now _getCurrentSupply() subtracts excluded owner:

```
tSupply = 950 - 100 = 850
rSupply = 950,000 - 50,000 = 900,000
New rate = 900,000 / 850 ≈ 1058.82 (not 1,000)
```

Recommendation

Include a conditional check to ensure the _tOwned variable is also decremented when excluded accounts are updated in a manual burn.

Medium Severity Issues

Trapped ETH in contract due to swap—and—liquify price impact mismatch

Resolved

Path

Token.sol

Function Name

`swapAndLiquify()`

Description

In swapAndLiquify, half the token balance is swapped to ETH, which moves the token price downward. When the remaining half is paired with the received ETH via addLiquidityETH, the router requires less ETH than was received (due to the now—lower token price). The surplus ETH remains in the contract. There is no withdrawal function, so this ETH accumulates permanently with each swap—and—liquify cycle.

Impact

This leads to accumulated dust locked after every liquidity addition event, this can lead to a non-trivial amount getting stuck and unable to be removed from the smart contract.

Likelihood

This is guaranteed to happen on every swap cycle.

Recommendation

Add an onlyOwner sweep() function to withdraw accumulated ETH from the contract.

Informational Issues

Inconsistent naming convention for private functions

Resolved

Path

Token.sol

Description

Some private functions use _ prefix (e.g., _reflectFee) while others don't (e.g., removeAllFee, restoreAllFee).

Recommendation

Standardize all private functions with the _ prefix.

Unused return values in addLiquidity

Resolved

Path

Token.sol

Function Name

`addLiquidityETH()`

Description

addLiquidityETH returns (amountToken, amountETH, liquidity) but the contract discards all three.

Recommendation

Capture and validate return values or document the intentional discard.

Dead code in _tokenTransfer

Acknowledged

Path
Token.sol

Function Name
`_tokenTransfer()`

Description
The final else branch (L1668) is unreachable because the four if/else if conditions already cover all boolean combinations of _isExcluded(sender) and _isExcluded(recipient).

Recommendation
Remove the dead branch.

Zero slippage protection in swap and liquidity calls

Partially Resolved

Path
Token.sol

Function Name
`swapTokensForEth(), addLiquidityETH()`

Description
swapTokensForEth and addLiquidityETH use amountOutMin = 0 and amountMin = 0.

Recommendation
Set a reasonable minimum output based on expected amounts or an oracle price.

Uses string revert reasons instead of custom errors

Acknowledged

Path

Token.sol

Description

All require statements use string messages, which cost more gas than Solidity 0.8.4+ custom errors.

Recommendation

Replace with error declarations and revert statements.

Redundant SafeMath usage on Solidity 0.8.20

Acknowledged

Path

Token.sol

Description

SafeMath is unnecessary since 0.8.0 has built-in overflow checks.

Recommendation

Remove the library and use native arithmetic operators.

Missing events for fee and state changes

Acknowledged

Path

Token.sol

Description

setTaxFeePercent, setLiquidityFeePercent, setCharityFeePercent, and excludeFromFee do not emit events.

Recommendation

Add events for all admin state changes to support off-chain monitoring.

Functional Tests

Some of the tests performed are mentioned below:

- ✓ Transfer to address(0) reverts
- ✓ Approve to address(0) reverts
- ✓ transferFrom reduces allowance by exact transfer amount
- ✓ Blacklisted sender and receiver both revert on transfer
- ✓ Blacklisted sender cannot bypass via transferFrom with prior approval
- ✓ Subsequent manual burns need a 72-hour timelapse
- ✓ Should update _tOwned for excluded accounts during a manual burn
- ✗ Should withdraw trapped native token from leftover dust in swapAndLiquify call

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
UniswapV2 Router	Router contract	Handles token swaps, liquidity additions	Official Uniswap Router address used	Slippage bypass, Frontrunning

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
Token.sol	constructor()	**What this function can do** Sets ERC20 state variables, tax-charity-liquidity addresses and values, initializes the router and excludes the owner and token contract from paying fees. **What this function cannot/should not do** Leave addresses or taxes unset **Main invariant(s)** Total supply is fixed at deployment **Access level** Implicit restriction (once at deployment)
Token.sol	manualBurn()	**What this function can do** Takes `tAmount` of the owner's tokens out of circulation and updates the balances of reflected and non-reflected tokens (_rTotal and _tTotal). **What this function cannot/should not do** Adjust the rate gotten by comparing the current supply via _getRate() **Main invariant(s)** _rOwned,_ tOwned, rSupply, tSupply, _rTotal, _tTotal **Access level** Only owner

Contract Name	Function	Category
Token.sol	deliver()	**What this function can do** Redistributes value to other included token holders via the reflection rate by reducing rTotal and increasing _tFeeTotal. **What this function cannot/should not do** Burn excluded token holders tokens **Main invariant(s)** _rOwned, _tFeeTotal **Access level** Only owner
Token.sol	transfer(), transferFrom()	**What this function can do** Transfers `amount` to the recipient from an address with sufficient approvals, adjusting state dependent on whether the accounts operated on are included or excluded from fees. **What this function cannot/should not do** Transfer tokens and bypassing the fees **Main invariant(s)** rOwned, tOwned, rSupply, tSupply, rTotal, tTotal **Access level** Only owner

Contract Name	Function	Category
Token.sol	removeAllFee(), restoreAllFee()	**What this function can do** Pair-wise functions used to cache fee values during transaction execution and restore immediately after execution. **What this function cannot/should not do** Change cached values before the end of transaction execution **Main invariant(s)** _previousTaxFee, _previousLiquidityFee, _previousCharityFee, _taxFee, _charityFee, _liquidityFee **Access level** Only owner

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
LiquidityGeneratorToken	ERC20	Liquidity fee tokens accumulated via _takeLiquidity pending swapAndLiquify
LiquidityGeneratorToken	ETH	ETH received from router swaps, held temporarily during swapAndLiquify; residual dust permanently trapped
uniswapV2Pair	ERC20 + WETH	Paired liquidity reserves (token + WETH); LP tokens minted to 0xdead are irrecoverable
_charityAddress	ERC20	Charity fee tokens credited via _takeCharityFee on every taxed transfer

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
LiquidityGeneratorToken	constructor	ETH (msg.value)	ETH to serviceFeeReceiver_	Deployer	Reverts if receiver cannot accept ETH
LiquidityGeneratorToken	transfer / transferFrom	Tokens from sender	Tokens to recipient; liquidity fee tokens to contract; charity fee tokens to _charityAddress	Any user	Fee-exempt if sender/receiver in _isExcludedFromFee; triggers swapAndLiquify if threshold met
LiquidityGeneratorToken	_takeLiquidity	–	Tokens credited to contract's _rOwned/_tOwned	Internal (via transfer)	Accumulates tokens for auto-LP; no external call
LiquidityGeneratorToken	_takeCharityFee	–	Tokens credited to _charityAddress _rOwned/_tOwned	Internal (via transfer)	Emits Transfer event; no external call
LiquidityGeneratorToken	_takeCharityFee	–	Tokens credited to _charityAddress _rOwned/_tOwned	Internal (via transfer)	Emits Transfer event; no external call

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
LiquidityGeneratorToken	swapAndLiquify	Tokens (half swapped)	ETH received from router swap; tokens + ETH sent to router for LP	Internal (via _transfer)	Zero slippage protection; ETH dust remains trapped; guarded by lockTheSwap
LiquidityGeneratorToken	swapTokensForEth	Tokens to router	ETH from router to contract	Internal (via swapAndLiquify)	amountOutMin = 0; router approval set per call
LiquidityGeneratorToken	addLiquidity	Tokens + ETH to router	LP tokens to 0xdead	Internal (via swapAndLiquify)	amountMin = 0 for both; LP permanently locked
LiquidityGeneratorToken	manualBurn	–	Tokens destroyed (supply reduced)	Owner only	Does not update _tOwned if owner excluded; 72h cooldown; no external call
LiquidityGeneratorToken	deliver	Tokens from caller (reflected)	Redistributed to all included holders via rate change	Any non-excluded user	Reduces _rTotal; does not reduce _tTotal; no external call
LiquidityGeneratorToken	receive	ETH from router	–	Router (during swap)	No access control; any address can send ETH; no withdrawal function exists

Closing Summary

In this report, we have considered the security of GrowFitech. We performed our audit according to the procedure described above.

Issues of critical, Medium and Informational Severity was found during Audit. The GrowFitech Team resolved 4 issues, partially resolved 1 issue and acknowledged the others.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

March 2025

For

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com